

Aula 06 - Programação de Sistemas Embarcados

Descrição: Programação de microcontroladores de forma progressiva, partindo da sintaxe básica e gestão de memória até técnicas avançadas, como multitarefa e otimização de hardware. O objetivo é desenvolver o raciocínio lógico-computacional, conectando a teoria diretamente à resolução de desafios reais.

Nível 1 - Fundamentos do Microcontrolador, Sintaxe e E/S Digital

Tópico 1.1: Estrutura Básica de um Sketch (setup() e loop())

Conceito Detalhado: O microcontrolador (como o ATmega328P do Arduino UNO) atua como um cérebro eletrônico de alta velocidade, porém sem iniciativa própria, exigindo instruções estruturadas passo a passo. O código escrito na IDE do Arduino é chamado de Sketch (salvo com a extensão .ino). Ele é processado por um compilador (como o avr-gcc), transformado em binário e gravado na memória Flash.

Ciclo de Vida do Programa: A linguagem exige duas funções obrigatórias:

- **setup():** Executada uma única vez assim que a placa é energizada ou resetada. É voltada para as configurações iniciais de hardware (como definir modos de pinos e inicializar barramentos de comunicação).
- **loop():** Laço infinito principal que roda continuamente enquanto o circuito receber energia. Ao atingir a última instrução, o processador retorna imediatamente à primeira linha da função.

Regras Sintáticas: Instruções individuais devem terminar obrigatoriamente com ponto e vírgula (;). Blocos de código são delimitados por chaves {}. A linguagem é sensível a maiúsculas e minúsculas (case-sensitive). Comentários de linha única usam // e de múltiplas linhas usam /* */.

```
// Função de configuração: executa apenas uma vez
void setup() {
  // Configurações de hardware entram aqui
}
```

```
// Função de execução contínua: laço infinito
void loop() {
  // Código executado repetidamente
}
```

Tópico 1.2: Variáveis, Constantes e Tipos de Dados Primitivos

Conceito Detalhado: O Arduino UNO possui apenas 2 KB de memória SRAM. Para efeito de comparação, um smartphone comum possui 4 milhões de vezes mais memória. Por isso, a escolha do tipo de dado correto é uma questão estratégica de sobrevivência para o sistema. Uma variável é um local nomeado na memória SRAM reservado para armazenar dados temporários.

Regras de Identificadores: Nomes de variáveis não podem começar com números, não aceitam espaços nem caracteres especiais (exceto o sublinhado) e seguem a convenção camelCase.

Tipo de Dado	Tamanho	Faixa de Valores / Uso
void	-	Tipo indefinido ou sem retorno de valor.
boolean / bool	1 byte	Armazena estritamente true (1) ou false (0).
char / byte	1 byte	char: caracteres ASCII. byte: Inteiro sem sinal (0 a 255).
int / unsigned int	2 bytes	-32.768 a +32.767 (int padrão) / 0 a 65.535 (unsigned).
long / float	4 bytes	Essencial para grandes contagens ou precisão decimal.

Estouro de Memória (Overflow): Se somarmos 1 a uma variável de tipo byte com valor 255, ela estoura seu limite e retorna a 0.

Constantes e Escopo: Constantes imutáveis são criadas com `const` ou via diretiva `#define`. Variáveis globais são declaradas fora de funções e persistem por todo o programa; variáveis locais são criadas dentro de funções e destruídas ao final da execução do bloco.

```
const float PI_CIRCUITO = 3.14159; // Constante em memória
int contadorLeituras = 0;          // Variável global de 2 bytes
```

```
void setup() {
  byte pinoSensor = 7;           // Variável local de 1 byte
}
void loop() {}
```

Tópico 1.3: Operadores Aritméticos, Relacionais e Lógicos

Conceito Detalhado: Operadores são ferramentas de processamento do microcontrolador para transformar dados. A divisão entre inteiros descarta decimais. O compilador aplica a Avaliação por Curto-Circuito: se a primeira condição de um operador lógico E (&&) for falsa, a segunda nem é processada.

- **Aritméticos:** +, -, *, /, % (módulo). Incremento (++) e atribuições compostas (+=, etc).
- **Relacionais:** == (igual a), != (diferente de), <, >, <=, >=. Retornam estritamente true (1) ou false (0).
- **Lógicos:** && (E lógico), || (OU lógico), ! (NÃO).

Atenção: Usar `=` no lugar de `==` dentro de um teste condicional força o resultado a ser verdadeiro. Erro comum!

```
int a = 10;
int b = 3;
void setup() {
  int resto = a % b;           // Resto da divisão: 1
  bool status = (a > 5) && (b < 5); // Avalia como true (1)
}
void loop() {}
```

Tópico 1.4: Funções de E/S Digital e Comunicação Serial (UART)

Conceito Detalhado: Pinos digitais operam exclusivamente em HIGH (5V) ou LOW (0V). A função `pinMode()`

define OUTPUT ou INPUT.

Integração Prática de Circuitos:

O modo INPUT_PULLUP é fundamental: ele ativa resistores internos (20 a 50 kΩ), eliminando o estado "flutuante" de ruídos eletromagnéticos sem exigir resistores externos na protoboard. Uma excelente prática de economia de hardware.

A comunicação UART permite troca com o PC via RX/TX. Inicia com Serial.begin(9600) e usa Serial.available() e Serial.read() para manipular o buffer de 128 bytes.

```
void setup() {  
  pinMode(13, OUTPUT);  
  pinMode(2, INPUT_PULLUP); // Botão com resistor interno ativado  
  Serial.begin(9600);  
}  
  
void loop() {  
  int estadoBotao = digitalRead(2); // Lê o botão (LOW se pressionado)  
  digitalWrite(13, !estadoBotao); // Acende o LED ao pressionar  
  Serial.println(estadoBotao);  
  delay(100);  
}
```

Nível 2 - Estruturas de Controle, Sinais Analógicos e Vetores

Tópico 2.1: Estruturas de Controle Condicional

Conceito Detalhado: Desviam o fluxo do programa para decisões autônomas. Incluem blocos if, else if e else.

Estrutura switch...case (Aplicação em Projetos): Ideal para menus de interfaces seriais em sistemas robóticos. Funciona comparando inteiros ou chars. Lembre-se do *Fall-Through*: se omitir o break, executa todos em cascata.

```
char comando = 'A';  
void setup() {  
  Serial.begin(9600);  
  switch (comando) {  
    case 'A':  
      Serial.println("Motor Ligado");  
      break; // Impede cascata  
    case 'B':  
      Serial.println("Motor Desligado");  
      break;  
    default:  
      Serial.println("Comando Invalido");  
      break;  
  }  
}  
  
void loop() {}
```

Tópico 2.2: Laços de Repetição

Conceito Detalhado: Automatizam repetições.

- **for:** Iterações conhecidas. for (inicialização; condição; incremento).
- **while:** Avalia no início. Cuidado com laços infinitos travando a placa.
- **do...while:** Avalia no final. Garante execução mínima de 1x (ideal para rotinas de calibração de sensores).

Tópico 2.3: Conversão Analógica, PWM e Funções Matemáticas

Conceito Detalhado: O mundo real é analógico; microcontroladores são digitais.

Entrada ADC: Conversor de 10 bits. Lê de 0V a 5V mapeando para 0 a 1023. `analogRead()`.

Saída PWM: Gera onda quadrada para simular tensão. Escala 8 bits (0 a 255). `analogWrite()`.

A função `map()` é vital para converter proporcionalmente a leitura do sensor (10 bits) para a saída do atuador (8 bits).

```
void setup() { pinMode(9, OUTPUT); }  
void loop() {  
    int valorLeitura = analogRead(A0);  
    int valorPWM = map(valorLeitura, 0, 1023, 0, 255);  
    analogWrite(9, valorPWM);  
}
```

Tópico 2.4: Coleções de Dados e Texto

Conceito Detalhado: Vetores (arrays) agrupam grandes dados. O índice começa em 0. Cuidado com a invasão de memória ao acessar índices inexistentes.

Para texto, evite a Classe String que fragmenta a RAM de 2KB. Prefira **C-Strings** (vetores de char terminados em `\0`).

Nível 3 - Modularização, Temporização Não-Bloqueante e Ponteiros

Tópico 3.1: Modularização e Modificador static

Conceito Detalhado: Dividir código em funções (com parâmetros e retornos). O modificador static preserva o valor da variável local entre sucessivas execuções, evitando o excesso de variáveis globais.

Tópico 3.2: Temporização Não-Bloqueante (Multitarefa Cooperativa)

Aplicação em Projetos de Robótica: A função `delay()` congela todo o processador. Em robótica, isso é inaceitável (um robô não pode ignorar um sensor de obstáculo enquanto pisca um LED).

Ao usar `millis()` (que retorna o tempo desde a energização em ms, durando ~49 dias até o overflow), capturamos "carimbos de tempo". Isso permite executar tarefas paralelas assíncronas.

```
unsigned long tempoAnterior = 0;
const long intervalo = 1000;
bool estadoLED = LOW;
void setup() { pinMode(13, OUTPUT); }
void loop() {
    unsigned long tempoAtual = millis();
    if (tempoAtual - tempoAnterior >= intervalo) {
        tempoAnterior = tempoAtual;
        estadoLED = !estadoLED;
        digitalWrite(13, estadoLED);
    }
    // CPU totalmente livre para ler sensores ultrassônicos simultaneamente!
}
```

Tópico 3.3: Ponteiros e Estruturas (struct)

Conceito Detalhado: Ponteiros armazenam o endereço físico SRAM de outra variável. Consomem 2 bytes. O operador & captura o endereço, e o * acessa o conteúdo.

Estruturação Profissional: Utilizando structs, o programador pode agrupar dados heterogêneos sob um identificador. Extremamente útil para organizar as múltiplas leituras de sensores complexos de um sistema em um pacote só.

```
struct SensorData {
    int id;
    float temperatura;
};
void zerarSensor(SensorData *s) {
    s->temperatura = 0.0; // Uso de seta para acessar membro via ponteiro
}
void setup() {
    Serial.begin(9600);
    SensorData s1 = {1, 25.5};
    zerarSensor(&s1); // Passagem por referência
}
void loop() {}
```

Nível 3 Avançado - Pílulas de Conhecimento Sob Demanda

Tópico 3.4: Manipulação de Bits (Bitwise) e Registradores de Deslocamento

Conceito Detalhado: Permite manipular individualmente os bits de dados em registradores de hardware com velocidade e eficiência máxima de processamento.

Operadores Bit a Bit:

- **& (AND Bitwise):** Retorna 1 apenas se ambos os bits forem 1.
- **| (OR Bitwise):** Retorna 1 se ao menos um dos bits for 1.
- **^ (XOR Bitwise):** Retorna 1 se os bits forem diferentes.

- **~ (NOT Bitwise):** Inverte todos os bits de uma variável.
- **<< (Deslocamento à Esquerda):** Desloca bits para a esquerda, multiplicando por potências de 2.
- **>> (Deslocamento à Direita):** Desloca bits para a direita, dividindo por potências de 2.

Máscaras de Bits e Funções Nativas: Máscaras combinam operadores para ligar (reg |= (1 << bit)), desligar (reg &= ~(1 << bit)) e testar bits. O Arduino abstrai essas ações pelas funções nativas bitSet(), bitClear(), bitRead() e bitWrite().

Expansão de Saídas: Permite controlar múltiplos pinos externos usando poucos pinos do microcontrolador por meio de CI registradores de deslocamento (como o 74HC595) usando a função shiftOut(pinoDados, pinoClock, ordemBits, valor).

```
byte registradores = 0b00000000; // Byte inicializado
void setup() {
  Serial.begin(9600);
  bitSet(registradores, 3); // Liga o bit na posição 3 -> 0b00001000
  Serial.println(registradores); // Exibe o valor numérico 8
  bitClear(registradores, 3); // Desliga o bit na posição 3 -> 0b00000000
}
void loop() {}
```

Tópico 3.5: Uso de Bibliotecas Externas e Gerenciamento de Memória (EEPROM e PROGMEM)

Inclusão de Bibliotecas: A diretiva #include <biblioteca.h> incorpora códigos prontos que abstraem periféricos complexos. Arquivos .h (header) declaram funções e classes; arquivos .cpp contêm as implementações.

Memória EEPROM Interna: Memória não-volátil onde os dados persistem mesmo após desligar a placa da tomada. O ATmega328P do Arduino UNO possui 1 KB (1024 bytes) de EEPROM. Utiliza a biblioteca <EEPROM.h> e as funções EEPROM.read(endereço) e EEPROM.write(endereço, valor).

Memória Flash (PROGMEM): A memória RAM do Arduino UNO é escassa (2 KB), enquanto a memória Flash possui 32 KB. Grandes tabelas estáticas, constantes fixas ou matrizes de fontes e texto causam o rápido esgotamento da RAM. Utilizando a palavra-chave PROGMEM da biblioteca <avr/pgmspace.h>, forçamos o armazenamento dessas tabelas estáticas diretamente na Flash, liberando a RAM. Os dados são lidos na execução através da função pgm_read_byte().

```
#include <EEPROM.h> // Biblioteca para EEPROM
#include <avr/pgmspace.h> // Biblioteca para PROGMEM
// Armazena vetor diretamente na memória Flash para economizar RAM
const byte tabelaMensagens[] PROGMEM = {10, 20, 30, 40, 50};
void setup() {
  Serial.begin(9600);
  EEPROM.write(0, 150); // Grava o valor 150 no endereço 0 da EEPROM
  int valorSalvo = EEPROM.read(0); // Lê o dado mantido sem energia
  // Lê o primeiro elemento da tabela gravada na memória Flash
  byte dadoFlash = pgm_read_byte(&(tabelaMensagens[0]));
}
void loop() {}
```

Tópico 3.6: Interrupções de Hardware (ISR)

Conceito Detalhado: Eventos urgentes que pausam a rotina normal instantaneamente (Pinos D2 e D3 do UNO) usando `attachInterrupt()`. Acionados por LOW, CHANGE, RISING ou FALLING.

Regras Rígidas para ISR:

- As funções ISR devem ser ultrarrápidas.
- Não usar comunicações seriais.
- **Proibido usar `millis()` ou `delay()`.** (Prática Recomendada: Adicione intencionalmente um `delay()` em uma ISR e observe o hardware travar de forma didática).
- Variáveis modificadas dentro da ISR devem possuir o modificador `volatile`.

Tópico 3.7: Uso de Macros com Pré-Processamento

O Papel do Pré-processador: O pré-processador da linguagem C atua antes da compilação real. Ele realiza substituição textual pura e simples. As diretivas iniciam com o caractere `#` (ex.: `#define`, `#include`, `#ifdef`).

Conceito Detalhado: O pré-processamento é a primeira etapa do processo de compilação. Nesta fase, o código-fonte é lido e modificado com base em diretivas, incluindo a remoção de comentários e a execução de trocas textuais antes da geração do código assembly ou binário.

- **`#define` (Expansão de Macros):** Permite definir um identificador que será substituído por um texto ou valor antes da compilação. Um erro comum é achar que o `#define` cria variáveis na memória; na verdade, atua como uma substituição textual e não cria "constantes" alocadas na SRAM do microcontrolador. Isso permite economizar memória.
- **`#include` (Inclusão de Arquivos):** Utilizada para incorporar o conteúdo de arquivos externos (como `.h` de bibliotecas ou configurações de hardware) diretamente no código.
- **`#ifdef` / `#ifndef` (Compilação Condicional):** Avaliam se uma macro foi definida ou não, permitindo que blocos inteiros de código sejam incluídos ou removidos do programa final. Isso é vital para criar diferentes versões do mesmo código base, ativando funcionalidades apenas quando o hardware suportar, economizando memória do sistema embarcado.

Integração Prática (Metodologia PBL):

O ensino das macros é fundamental na programação de robôs. Por exemplo, usar `#define PINO_MOTOR 9` ao invés de usar uma variável `int` tradicional preserva a escassa memória SRAM do Arduino e facilita a legibilidade e a manutenção do hardware do projeto.

```
#define PINO_LED 13    // Substituição textual; não ocupa SRAM
#define MODO_DEBUG    // Define uma macro para ativar o modo de depuração

void setup() {
    pinMode(PINO_LED, OUTPUT);
    Serial.begin(9600);
}

void loop() {
    #ifdef MODO_DEBUG
        // Compilado e enviado para a placa apenas se MODO_DEBUG estiver definido
        Serial.println("Executando loop de controle...");
    #endif
}
```